Product-share encoding: the efficiency and hardening pass

What changed, and why

2026-07-24

Assumes the design in `docs/PRODUCT_SHARE_ENCODING`. *For the construction described from scratch, see* `docs/NONLINEAR_GADGETIZATION`.

1 Summary

The encoding worked—it was the first method to kill the computational-progress diagonal—but it was expensive (125× the source circuit) and it carried two weaknesses we had not looked at, both outside the reconstruction-heatmap's field of view. This pass addresses cost and both weaknesses. The production gadget is now **25% smaller on 8 fewer wires with identical measured security**, its band no longer supplies free affine invariants to a SAT attacker, and the fold no longer advertises where each source gate begins and ends.

	shipped (07-23)	now (07-24)
plan	$[2, 2, 3, 3]$	$[\mathbf{2}, \mathbf{3}, \mathbf{3}]$
band fill	linear $\langle \alpha, x \rangle$	**nonlinear cascade + mirror** F'
n=128 gadget	374,478 gates / 576 wires	**281,068 / 568**
deg-1 ridge	depthMed 0.000, ρ 0.017	depthMed 0.000, ρ 0.015
deg-2 ridge	depthMed 0.000, ρ −0.035	depthMed 0.000, ρ −0.007

A fourth change—a mode that removes wide gates entirely—was built and measured, and the measurement says **do not use it**. That negative result is §4, and it is the most useful thing in this document.

2 $[2, 3, 3]$: drop one base term

k (term count) and deg (literals per term) do different jobs. Degree hides **algebraically**: a degree-d mask sits outside the degree-$< d$ GF(2) span, so *one* such term already forces $H = 0.5$ against every adversary of degree $< d$, whatever k is. k hides **statistically**, by piling-up: k stacked terms push the best affine readout error toward $1/2$ as $1/2 - 2^{-(k+1)}$.

So the two degree-3 tower terms are what kill the degree-2 diagonal, and the *second* degree-2 base term buys nothing algebraic—only statistical margin. Dropping it ($[2, 2, 3, 3] \to [2, 3, 3]$) costs **24% of the gates** and moves the best-affine agreement from 0.57 to 0.64.

The honest caveat. That statistical cost is invisible to our instrument: the exact-span ridge measure saturates at $H = 0.5$ for any $k \geq 1$, so it scores $[2, 3, 3]$ and $[2, 2, 3, 3]$ identically—as it did. Building a statistical (best-approximation) readout is the top open item; until it exists, every k-reduction is being judged by a measure blind to exactly what k buys.

3 Nonlinear band fill (and a mirror at the output)

The weakness. The mask sources live on a frozen band, filled at the input port with $band_j = \langle \alpha_j, x \rangle$—a *linear* function of the inputs. Every band wire was therefore a **learnable affine invariant of the whole circuit**, held from the input port to the output port. That is precisely the class of relation the invariant-injection experiment showed **collapses the preimage SAT search**: forward-learned GF(2) invariants turned n=32 instances from ">2h timeout" into "∼10 minutes". We were manufacturing them by construction.

The fix. The fill becomes

$$band_j \;=\; junk \;\oplus\; x_{\mathrm{pivot}_j} \;\oplus\; (\text{small linear part}) \;\oplus\; \bigoplus^{M} (\text{two-source products})$$

where each product's two sources are drawn from the data wires **and from already-filled band wires**. The cascade is what makes this cheap: multiplying two already-balanced band bits keeps the firing rate near 1/4 while the *input degree* multiplies up the band, so high degree in x is reached without a single wide gate—every fill gate is a CNOT or a g57. (A flat high-degree monomial would instead fire on a 2^{-d} fraction of inputs and degenerate back to its linear part.)

Balance is preserved exactly, which the mask-balance argument needs: the fresh pivot x_{pivot_j} is excluded from the rest of that wire's transitive support, so the fill is a balanced function of x for *any* choice of the nonlinear part. This is asserted directly in the test suite.

Mirror F'. The same fill is now emitted again after the output bookend. The band is junk at both ports, so a two-sided composition no longer sees it anchored only at its far end.

Cost: **free** (281,068 gates with the fill versus 281,096 without—the difference is draw noise). The plate also improves: meanH 0.4931 → 0.4994, stdH 0.036 → 0.016.

What this does not fix. Band wires are still *body-static*: never written between the two ports, and heavily read. They remain trivially identifiable statically, which is what a restriction adversary needs in order to condition on one. The real fix is a **rolling band**—retire a band wire mid-body, migrate its live slots with the existing re-source machinery, refill it from current carrier values—which keeps the frozen-invariance property piecewise. Not built.

4 Fold-order randomization

The weakness. Each source gate emitted its $(k_{\mathrm{total}}+2)^2$ fold fragments in deterministic odometer order, consecutively, all targeting the same value's two carriers, separated from the next fold by a few RG gates. That is a **progress clock requiring no execution at all**: an attacker segments the fold blocks by eye and reads off the source circuit's gate boundaries—the very thing the encoding exists to hide, handed over syntactically.

Fragments are now shuffled within each fold. This is the cheap half of the fix; the stronger version—interleaving fragments *across* adjacent folds through the existing randomized-insertion pass, plus

random ESOP re-covers and k-jitter so the per-fold fragment count is not constant—is designed but not built. Note that the final `commuting_shuffle` and downstream mixing already move fold material around; what was missing was randomization at the point of emission.

5 Narrow mode—built, measured, and *not* recommended

The motivating fear was concrete: the encoding's fold fragments reach width 6, `--db-ctrl-cap 2` makes every gate with ≥ 3 controls permanently ineligible for frozen-DB re-encoding, and an inert, uncrossable subpopulation would be a fossil signature that mixing could never erase.

`--prod-max-width W` removes wide gates entirely. The mechanism is the generalized **Barenco double sweep over dirty borrowed carriers**: a width-w conjunction becomes $\sim 4(w-2)$ two-control gates over $w-2$ borrowed wires whose contents are *arbitrary*, because each borrow is visited an even number of times and its dirty value cancels between readings. (The codebase already contained the w=3 instance, in `emit_poly_add`'s cubic case.) So it costs **zero extra wires**, leaves no wire sitting at a constant, and preserves the gadget's "correct under arbitrary junk on every non-data wire" contract.

Two implementation facts worth recording. Ladder rungs must be **exact**: a g57's complement on a borrowed wire does *not* cancel, it leaks a spurious $c \cdot \kappa$ into the target. And an exact 2-control conjunction is **not** a sum of g57s—every g57 carries a constant 1, so an odd number leaves a stray 1 and an even number collapses the monomials into a plain XOR. Rungs are therefore `comp=0` width-2 gates, which are still DB-eligible: `db_ctrl_cap` filters on width, not on `comp`.

The measurement that killed it. A per-width commutation-leeway census of the real n=128 gadget:

width	count	mean float-box	median
6	24,296	**2778**	**2426**
5	28,037	2779	2415
3	56,160	2218	1621
4	65,907	1627	1244
2 (g57)	41,934	**694**	**416**

The wide mask fragments are the **most mobile gates in the circuit**—4× the mean and 5.8× the median mobility of the g57s. This is the opposite-literal separation exemption: a gate with many literals has *more* chances to hold an opposite literal against a neighbour, hence more freedom to cross it. The premise was simply wrong. Narrowing then costs **6.4**× the gates (1.80M vs 281k at n=128) and drops rung mobility to mean 259 / median 96.

Security is unaffected either way—wide and narrow builds give identical ridge results at degree 1 and degree 2, and the $[2, 2]$ capability controls come back alive ($\rho = 1.00$) in *both*, so the nulls are genuine hiding and the ladder's transient partial products leak nothing.

Narrow mode's one remaining justification is **DB re-encodability**: wide gates can never be re-encoded, which is why generation targeting reports $G = 0$ forever (§6). Whether that is worth 6.4× the size is unmeasured; it needs a `--db-dry-run --db-record` match-rate A/B on narrow versus wide material. The flag stays in the tree, off by default.

6 Generation targeting: what "generation 100" can mean here

`G=` in the fmix report is the 5th-percentile gate generation **over all gates**. About 62% of a product-share gadget is width ≥ 3 and therefore cap-ineligible: those gates are never re-encoded, their generation stays 0, and `G=` is pinned at 0 **by construction**. "Run to generation N" cannot mean the reported `G=` on this material, and no amount of mixing will change that.

The achievable target is generation N **over the DB-eligible material**, which is what `--gen-target` actually drives and what `--gen-stop-frac` measures (lag/elig), with `--gen-giveup` writing off eligible gates the DB genuinely cannot reach so the dose stop can fire. Read `lag=/elig=` in the report; ignore `G=`. A live run shows the split cleanly:

```
gen tgt=100 G=0 alag=280020/283384 lag=105410/106226 wlag=174598 u=12
```

`wlag` is the wide population (invisible to the DB), lag/elig is the real progress number.

7 Status

Committed on `ssg-gen-mix-clean`. Library suite **171/171** (five new tests: g57-form exactness over all polarity combinations; ladder exactness and borrow restoration over the full input domain; share-native narrow fold; full narrow gadget round-trip; band-fill balance and nonlinearity). Endpoint verification passes at $n = 4, 16, 128$.

Open, in priority order: the statistical readout (§2); the DB match-rate A/B (§5); deeper fold randomization (§4); the rolling band (§3).